

Code Metrics Dataset - Yapay Zeka Modelleme Stratejisi

Ders: Yazılım Test Sürecinin Optimizasyonu İçin Yapay Zeka Yöntemleri

Bu belge, analiz edilen **Code Metrics Dataset (SoftwareProjectStructure)** veri seti üzerinden makine öğrenmesi ve yapay zeka modelleri geliştirmek için izlenebilecek stratejileri, uygun hedef değişkenleri, eğitim adımlarını ve test optimizasyonu uygulama örneklerini açıklamaktadır.

1. Hedef Değişken Belirleme (Target Variable)

Ana Hedef: **refactoring** (İkili Sınıflandırma - Binary Classification)

Veri setinde modelleme için en uygun ve halihazırda etiketlenmiş olan hedef değişken **refactoring** sütunudur. Hedefimiz: *Bir Java sınıfının sahip olduğu karmaşıklık, satır sayısı, bağımlılık (CBO, WMC, LOC vb.) gibi nesne yönelimli tasarım metriklerine bakarak, o sınıfın refactoring (yeniden yapılandırma) işlemine ihtiyaç duyup duymadığını tahmin etmektir.*

- Sınıflar: **0 (Refactoring Yapılmamış)**, **1 (Refactoring Yapılmış / İhtiyaç Var)**

Alternatif Senaryolar

Veri setinde **type** (class, interface, enum) ve **dataset** (Trivial / NonTrivial) sütunları bulunmaktadır. Sadece *Non-Trivial (Kritik)* refactoring işlemlerine odaklanan ayrı bir model eğitmek, projenin teknik borç (technical debt) önceliklendirmesi için daha değerli olabilir.

2. Kullanılabilecek Model Türleri

Analiz raporunda (Bölüm 3 ve 9), metriklerin aşırı yüksek çarpıklığa (skewness) ve yüksek oranda uç değerlere (**lcom**'da %16, **loopQty**'de %14) sahip olduğu görülmüştür. Ayrıca sınıf dengesi 1:1'dir (Dengeli).

1. Ağaç Tabanlı Topluluk Modelleri (Yüksek Öneri) 🌳

- Random Forest:** Aşırı yüksek (VIF=188K) çoklu doğrusal bağıntı (multicollinearity) bulunan bu veri setinde, özellik seçimi (feature selection) açısından en toleranslı algoritmadır. Outlier'lardan etkilenmez.
- XGBoost & LightGBM:** 741,000 satırlık bu devasa veri setinde, eğitim hızı ve performansı açısından en ideal seçeneklerdir. Dağılımın çarpık olması XGBoost için problem yaratmaz.

2. Derin Öğrenme (Deep Learning) Modelleri 🧠

- TabNet veya MLP:** Veri boyutu çok büyük olduğu için derin öğrenme modelleri (Multi-Layer Perceptron) çok başarılı olabilir. Ancak eğitim öncesi **RobustScaler** ile aykırı değerlerin mutlaka ölçeklenmesi gereklidir.

3. Mesafe Tabanlı Modeller (Önerilmez) ✖

- K-NN veya Support Vector Machines (SVM), verideki aykırı değerler ve 740K+ satırlık veri hacmi nedeniyle aşırı yavaş çalışacak ve performans kaybı yaşatacaktır.

3. Eğitim Adımları ve Stratejiler

Adım 1: Veri Ön İşleme (Preprocessing) & Özellik Seçimi (Feature Selection)

Analiz raporundaki VIF ve ANOVA testlerine göre veri temizliği şarttır:

- **Kritik Çıkarım:** `cbo` ve `fanout` sütunlarının korelasyonu 1.0, VIF değerleri 188,000'dir. Tamamen aynı şeyi ölçmektedirler. Modele girmeden önce **fanout** sütunu kesinlikle drop edilmelidir (silinmelidir).
- **Eksik Veri (Missing Values):** `tcc` ve `lcc` sütunlarında %32 eksik veri vardır. Bu sütunlar ya silinmeli ya da KNN Imputer / Medyan Imputer ile doldurulmalıdır.
- **Outlier İşleme:** Doğrusal veya sinir ağı modeli denenecekse `RobustScaler` zorunludur. Ağaç tabanlı modeller için ise direkt kullanılabilir.

Adım 2: Model Değerlendirmesi İçin Veri Bölünmesi (Train-Test Split)

Sınıf dengesi (%50.9 vs %49.1) neredeyse mükemmeldir. Bu yüzden **SMOTE uygulanmasına gerek YOKTUR**. Veri hacmi çok yüksek olduğu için %80 Eğitim (Train), %10 Doğrulama (Validation - Hiperparametre optimizasyonu için), %10 Test seti oluşturulması idealdir.

Adım 3: Çoklu Doğrusallığı Önleme (Multicollinearity Handling)

`wmc`, `loc` ve `totalMethodsQty` arasında yüksek korelasyon (0.82+) bulunmaktadır. Sadece Random Forest veya XGBoost kullanmıyorsanız, bu özelliklerden boyut indirgeme amacıyla PCA (Principal Component Analysis) çıkarımları veya `corr() > 0.85` filtresi (Feature Elimination) kullanılarak veri sadeleştirilmelidir. Modelin kompleksliğini düşürmek, açıklanabilirliği (XAI) artırır.

Adım 4: Değerlendirme Metrikleri

Veri dengeli olduğu için **Accuracy (Doğruluk)** ve **ROC-AUC** metrikleri referans olarak alınabilir. `refactoring` ihtiyacını kaçırmamak önemliyse (False Negatifleri azaltmak), **Recall (Duyarlılık)** skoru optimize edilmelidir.

4. Test Optimizasyonu Çerçevesinde Uygulama Örnekleri (Use Cases)

Eğitilen AI modeli, Yazılım Test Sürecini optimize etmek için aşağıdaki senaryolarda kullanılabilir:

Senaryo 1: Smart Regression Testing (Akıllı Regresyon Testi) 🔄

Problem: Binlerce sınıfı olan bir Java projesinde küçük bir değişiklik yapıldığında uçtan uca tüm testlerin koşulması saatler alır. **Yapay Zeka Çözümü:** Sınıfların mevcut kod metrikleri modele verilir. Eğer model bir sınıfın "**Yüksek Refactoring İhtiyacı**" (1) olduğuna kanaat getirirse, bu sınıfın karmaşık ve kırılmaya (bug) yatkın olduğu varsayılır. CI/CD pipeline'larında yalnızca refactoring ihtiyacı skoru %70 ve üzeri olan sınıfların unit/integration testleri öncelikli olarak çalıştırılır. Bu sayede regresyon testi süresi %80 oranında optimize edilir.

Senaryo 2: Technical Debt (Teknik Borç) Barometresi ve Sprint Planlama 📊

Problem: Proje yöneticileri hangi kodların acil refactor edilmesi gerektiğini göremediği için test aşamasında sürekli entegrasyon hataları patlar. **Yapay Zeka Çözümü:** Model her gece codebase'i (kaynak kodu) tarar. XGBoost modelinin "Feature Importance" (Özellik Önemi) analizine göre, örneğin `maxNestedBlocksQty` (iç içe

çok fazla if/else bloğu) değişkeninden dolayı refactoring uyarısı veren sınıfları listeler. QA ve Test ekibi, bu listeye bakarak *"Sistemde şu an X, Y, Z modüllerinde spagetti kod var, bir sonraki sprint'te testçilerin bu sınıflara odaklanması (Exploratory Testing) gerekiyor"* kararı alabilir.

Senaryo 3: AI-Destekli Code Review Botu (GitHub/GitLab Entegrasyonu)

Problem: Kod inceleme süreçleri manuel ve yavaştır; yazılımcılar karmaşık sınıfları fark etmeden "Merge" (birleştirme) edebilir. **Yapay Zeka Çözümü:** Eğitilen model, bir PR (Pull Request) açıldığında otomatik olarak devreye giren bir bot haline getirilir. Bot, yeni yazılan sınıfın **cboModified**, **loc**, **wmc** gibi metriklerini anlık olarak hesaplar. Eğitilmiş modele sorar. Eğer model "1" sonucunu döndürürse, bot yoruma (comment) şunu yazar: *"Uyarı: Eklediğiniz bu sınıf yapısal olarak refactoring gerektiren sınırları aşmıştır. Test ve bakım süreçlerinin optimize edilebilmesi için lütfen fonksiyonları parçalayınız."* Böylece daha test aşamasına gelmeden optimizasyon süreci "Shift-Left" yapılarak başa çekilir.